Projet : Questionnements géographiques en santé

L'offre de soins par département

Module	Format	Soutenance	Dépôt
Initiation à Streamlit	Projet de groupe	Lundi 30 novembre, 20 minutes par groupe	Sur Updago, avant la soutenance

Tout retard de dépôt vaut 0 à l'ensemble du groupe.

1. Le cas : pourquoi un zonier en assurance santé ?

Une mutuelle ou un assureur santé propose des complémentaires santé qui remboursent, en plus de l'Assurance Maladie, une partie des dépenses de soins de leurs assurés : consultations, optique, dentaire, hospitalisation...

Pour fixer le prix de ces contrats (la cotisation), l'assureur doit estimer ce que coûtera en moyenne un assuré. Or ce coût n'est pas le même partout en France :

- **L'offre de soins** varie fortement d'un territoire à l'autre : là où il y a beaucoup de médecins, de spécialistes, de dentistes ou d'hôpitaux, les assurés consultent plus facilement, donc plus souvent.
- **Les prix des soins** varient aussi : certains territoires concentrent davantage de praticiens qui pratiquent des dépassements d'honoraires.
- **La population** n'a pas le même profil partout (âge, besoins de santé).

> **Un zonier** est l'outil qui traduit ces différences dans le tarif : il regroupe les départements en quelques zones homogènes (par exemple zone 1 = coût nettement minoré, zone 3 = zone neutre, zone 5 = coût nettement majoré), et chaque zone reçoit un coefficient tarifaire.

Les enjeux pour l'assureur

- **Tarifer juste :** un tarif trop bas dans une zone coûteuse crée des pertes ; un tarif trop haut dans une zone peu coûteuse fait fuir les clients vers la concurrence.
- **Rester lisible :** quelques zones, faciles à expliquer aux clients et aux commerciaux, plutôt qu'un tarif différent par département.
- **S'appuyer sur des données objectives et publiques :** densité de professionnels, équipements, population... pour justifier le découpage.
- **Suivre les évolutions :** déserts médicaux, départs à la retraite, ouverture de centres de santé modifient l'offre d'année en année.

Votre mission

Construire une application Streamlit qui analyse l'offre de soins par département et qui aide à répondre à une question concrète, par exemple :

- Quels départements regrouper dans une même zone tarifaire, et pourquoi ?
- L'Île-de-France et les grandes métropoles forment-elles une zone à part ?
- Les départements les mieux dotés en spécialistes sont-ils aussi les mieux dotés en généralistes ?
- Où se trouvent les déserts médicaux, spécialité par spécialité ?
- Quelles spécialités sont les plus inégalement réparties sur le territoire ?

*Ces questions sont des exemples : chaque groupe formule **sa** question et la défend.*

2. Les sources de données

2.1 Le cas proposé : la Base permanente des équipements

Source	Contenu	Téléchargement
Base permanente des équipements (BPE) 2025 Insee	Un équipement par ligne, avec son département (DEP) et son type (TYPEQU). Le domaine D « Santé et action sociale » recense les professionnels libéraux (généralistes, ophtalmologues, dermatologues, ORL, gynécologues, dentistes, kinés, infirmiers...), les établissements (hôpitaux, urgences, maternités, centres de santé...) et les pharmacies.	Page de la BPE 2025 Fichier Parquet (163 Mo) Nomenclature des types
Populations de référence 2023 par département Insee	Indispensable pour calculer des densités (nombre pour 100 000 habitants). Utiliser le fichier des départements contenu dans l'archive.	Page des populations 2023 Archive ensemble.zip
Contours des départements Référentiel cartographique (GeoJSON)	Formes des départements pour la carte choroplèthe.	france-geojson Contours officiels (data.gouv.fr)

> **Attention :** la BPE compte des lieux d'exercice libéral. Un praticien qui exerce à deux adresses compte deux fois, et les médecins salariés des hôpitaux n'y figurent pas. Ces limites doivent être connues et discutées à l'oral.

2.2 D'autres sources officielles pour enrichir l'analyse

Source	Producteur	Intérêt
Accessibilité potentielle localisée (APL)	DREES	Accès aux médecins par commune, en tenant compte des distances et des besoins
Pathologies : effectifs de patients	Assurance Maladie (Cnam)	Besoins de santé par département : diabète, maladies cardiovasculaires, cancers...
FINESS : établissements	ANS	Établissements sanitaires et sociaux, avec leur localisation
Annuaire Santé (RPPS)	ANS	Professionnels de santé et leurs lieux d'exercice
Statistique annuelle des établissements de santé (SAE)	DREES	Lits, places et équipements des hôpitaux

2.3 Sujet libre

Le cas de l'offre de soins est proposé, mais le choix du sujet est libre, à condition :

- d'utiliser des sources officielles et ouvertes (Insee, data.gouv.fr, DREES, Assurance Maladie, ministères...) ;

- de respecter toutes les consignes de la partie 3 ;
- **de faire valider le sujet par l'enseignant au plus tard le lundi 16 novembre.**

> **Un sujet non validé à cette date ne sera pas accepté : le groupe traitera le cas proposé.**

3. Les consignes du projet Python

3.1 Données

- ☐ Croiser au minimum **2 sources de données différentes** (par une jointure), sans compter le référentiel des départements (contours, liste des codes et des noms). Exemple valide : BPE + population Insee.
- ☐ Contrôler chaque jointure : nombre de lignes avant et après, clés sans correspondance (attention aux codes « 01 » devenus « 1 », à la Corse 2A / 2B, à l'outre-mer 971 à 976).

3.2 Streamlit

- ☐ Utiliser au minimum **2 des 3 méthodes avancées** vues en cours :
  - `st.form` : regrouper des saisies validées par un bouton ;
  - `st.session_state` : mémoriser une valeur entre deux interactions ;
  - `st.cache_data` : ne pas relire ni recalculer les données à chaque clic.

  Chaque méthode utilisée doit avoir une vraie utilité, à expliquer à l'oral.
- ☐ Proposer des filtres (par département, région, type de professionnel...) et au moins une visualisation cartographique ou graphique pertinente.

3.3 Code

- ☐ Structurer le code de manière claire et segmentée : plusieurs fichiers si c'est pertinent (chargement des données, calculs, graphiques, affichage...).
- ☐ Écrire le code en fonctions (`def`), appelées depuis une fonction `main()`, elle-même lancée derrière `if __name__ == "__main__":`
- ☐ Documenter **chaque fonction** avec une docstring (rôle, arguments, valeur renvoyée).
- ☐ Rendre le projet **duplicable** : n'importe qui doit pouvoir le lancer sur son poste.
  - un fichier `requirements.txt` avec toutes les librairies nécessaires ;
  - un fichier `README.md` : présentation, données utilisées (sources, nom des participants du groupe, installation, lancement, organisation du code ;
  - **des chemins relatifs (aucun chemin du type `C:\Users\...`).**

3.4 Analyse

- ☐ Le projet doit permettre de **répondre à une question et de porter une analyse** : l'application n'est pas un simple catalogue de graphiques, elle sert un raisonnement (constats, chiffres clés, conclusion, limites).

3.5 Organisation

- ☐ Travail en groupe : **4 groupes de 4 étudiants et 1 groupe de 3 étudiants**.
- ☐ Les tâches sont réparties équitablement : chaque membre doit pouvoir expliquer la partie du code dont il a la charge, et présenter sa contribution à l'oral.

4. Le rendu

> **Le projet est déposé sur Updago avant la soutenance. Tout retard vaut 0 à l'ensemble du groupe.**

Contenu du dépôt :

- le code de l'application (tous les fichiers .py) ;
- le README.md et le requirements.txt ;
- les données, ou, si elles sont trop volumineuses, les liens et noms des fichiers dans le README ;
- le support de présentation (PPT).

> **Conseil :** avant de déposer, testez votre projet sur un autre poste que le vôtre, en suivant uniquement votre README.

5. La soutenance - Lundi 30 novembre

Chaque groupe dispose de 20 minutes, en trois temps :

Partie	Durée	Contenu attendu
1. Présentation avec un support PPT	≈ 6 min	L'enjeu traité et la question posée Les données utilisées et les jointures réalisées Les axes d'analyse La répartition des tâches dans le groupe Les techniques Streamlit utilisées, et pourquoi
2. Démonstration en direct	≈ 6 min	Utilisation de l'application en direct L'analyse qui en découle : réponse à la question, chiffres clés, limites
3. Questions / réponses sur le code	≈ 6 min	Chaque membre peut être interrogé sur n'importe quelle partie du code, par exemple : *« Pourquoi ce cache ici ? », « Que se passe-t-il au rechargement ? », « Comment est faite cette jointure ? », « Que fait ce if dans le session_state ? »*

6. Calendrier

Date	Étape
Mercredi 4 novembre	Lancement du projet, constitution des groupes, choix du sujet
Lundi 16 novembre	Séance de travail supervisée **Date limite de validation d'un sujet libre**
Lundi 30 novembre	**Dépôt sur Updago avant la soutenance** **Projet zippé + Présentation PPT : (avec vos noms !)** Soutenance (20 minutes par groupe)

7. Check-list avant le dépôt

- ☐ Au moins 2 sources croisées (hors référentiel des départements)
- ☐ Au moins 2 méthodes avancées parmi form, session_state et cache
- ☐ Code en fonctions, appelées par `main()`, derrière `if __name__ == "__main__":`
- ☐ Une docstring pour chaque fonction
- ☐ README.md et requirements.txt complets ; chemins relatifs
- ☐ Projet testé sur un autre poste en suivant le README
- ☐ Une question posée et une analyse qui y répond
- ☐ Support PPT : enjeux, axes d'analyse, répartition des tâches, techniques Streamlit
- ☐ Dépôt sur Updago fait avant la soutenance

Bon projet !